

FROM TOKENS TO WORDS: ON THE INNER LEXICON OF LLMs

Guy Kaplan, Matanel Oren, Yuval Reif, and Roy Schwartz

The Hebrew University of Jerusalem

{guy.kaplan3, matanel.oren, yuval.reif, roy.schwartz1}@email.huji.ac.il

ABSTRACT

Natural language is composed of words, but modern large language models (LLMs) process *sub-words* as input. A natural question raised by this discrepancy is whether LLMs encode words internally, and if so how. We present evidence that LLMs engage in an intrinsic detokenization process, where sub-word sequences are combined into coherent whole-word representations at their last token. Our experiments show that this process primarily takes place within the early and middle layers of the model. We further demonstrate its robustness to arbitrary splits (e.g., “cats” to “ca” and “ts”), typos, and importantly—to out-of-vocabulary words: when feeding the last token internal representations of such words to the model as input, it can “understand” them as the complete word despite never seeing such representations as input during training. Our findings suggest that LLMs maintain a latent vocabulary beyond the tokenizer’s scope. These insights provide a practical, finetuning-free application for expanding the vocabulary of pre-trained models. By enabling the addition of new vocabulary words, we reduce input length and inference iterations, which reduces both space and model latency, with little to no loss in model accuracy.¹

1 INTRODUCTION

Large language models (LLMs) rely heavily on tokenization methods such as byte-pair encoding (BPE; Sennrich et al., 2016). Such methods often split words into multiple tokens, potentially disrupting their morphological structure (Batsuren et al., 2024).² Typos and other perturbations can also lead to large variations in the tokens that represent a word (Kaushal & Mahowald, 2022). Nonetheless, LLMs exhibit a remarkable ability to recover word meaning (Cao et al., 2023). This raises important questions about how models internally compose meaningful word representations from tokens, a process referred to as *detokenization* (Elhage et al., 2022; Gurnee et al., 2023).

In this work, we seek to understand the detokenization mechanism in LLMs. We consider two cases: words that are not part of the model’s BPE vocabulary, and are thus split into multiple sub-words; and single-token, in-vocabulary words that we artificially split into multiple tokens. In both cases, our experiments indicate a word-level detokenization process in LLMs, which occurs mainly in the early to middle layers. Our results hint that language models hold a *latent vocabulary* or *inner lexicon*, which they access to identify words from token sequences.³

We begin by examining if internal representations of token sequences reflect whether or not a sequence of tokens comprises a word (Sec. 3). We probe the model’s hidden representations (Conneau et al., 2018) of both multi-token real English words and gibberish *nonwords* (Frisch et al., 2000). We observe that the representations of words and nonwords substantially diverge in middle layers—a simple k-nearest neighbors classifier achieves a 89% accuracy on the task of discriminating between the two groups. Overall, these results suggest that models hold a concept of recognized words.

We next explore the mechanism through which models reconstruct cohesive word representations from sub-word tokens (Sec. 4). To do so, we use techniques that interpret a token’s hidden states and

¹We release our code at <https://github.com/schwartz-lab-NLP/Tokens2Words>.

²For instance, the word “unhappiness” might be tokenized into “un,” “h,” and “appiness” (see Fig. 1)

³This process might resemble the *mental lexicon* in humans (Aitchison, 2012; Marslen-Wilson et al., 1994).

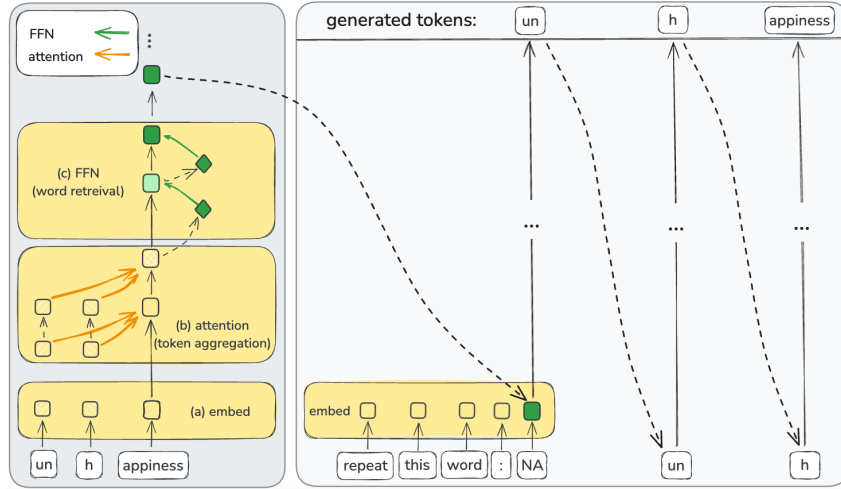


Figure 1: **Left:** The sub-word detokenization process in LLMs. From bottom to top: **(a) Tokenization and Embedding:** The input string is tokenized using a sub-word tokenizer (e.g., BPE) and converted into vector embeddings; **(b) Token Aggregation:** The attention mechanism relays information from the word’s previous tokens (“un”, “h”) into its final sub-word representation (“appiness”); **(c) Word Retrieval:** The model retrieves the full word representation from an implicit internal lexicon in its feedforward (FFN) layers. This representation is added to the residual stream, until it takes over the word’s hidden representation.

Right: When taking this hidden representation and patching it into another prompt, the model interprets it as the original word. In this example, the model is prompted to repeat the (single vector) hidden representation of the word “unhappiness” (originally represented as three tokens), and is able to “understand” it by regenerating the original three tokens.

decode them into natural language (Belrose et al., 2023; Ghandeharioun et al., 2024). We find that for both multi-token and single-token words, in most cases, the last token can be decoded as the full word after being processed by 3–5 layers, with some words requiring up to 15 layers. Interestingly, 23% of the multi-token words are never successfully decoded from their last token’s hidden states, hinting this inner lexicon does not cover all words.

We then turn to explore how models assemble these full word representations (Sec. 5). We first interpret the feedforward network (FFN) updates in the vocabulary space (Geva et al., 2021; Todd et al., 2024). We show that in 85% of the evaluated words, an FFN update promoting the full word’s concept is written to the last token’s residual stream (Geva et al., 2022; Merullo et al., 2024). Importantly, this happens in the layers *before* the full word’s representation emerges, indicating that models retrieve the reconstructed word representation from their FFN weights. Second, we compare attention patterns between standard single-token words to how multi-token words attend to their preceding sub-word tokens. We find that the last token in multi-token words attends significantly *more* to its previous tokens (which are sub-words of the same word) in layers 1–2 compared to single-token words (where these previous tokens represent other words), and significantly *less* in the following layers. This suggests that models initially aggregate information from previous tokens, and later almost ignore them. See Fig. 1 for an overview.

Our findings have concrete applications (Sec. 6); they allow us to expand the model input and output BPE vocabulary with the fused representation of multi-token words found in our experiments. Importantly, this expansion requires no update to model parameters. Applying our approach to multi-token words found in English Wikipedia data (Merity et al., 2017), we find that the model successfully uses the new vocabulary entries both as inputs and during generation: its language model performance is maintained, and even slightly improves. This demonstrates the potential to dramatically reduce both input and output sequence length, and inference costs accordingly, especially in languages where the ratio of tokens per word is high (Ahia et al., 2023; Petrov et al., 2023).

Overall, our results establish that word-level detokenization is a core process in LLMs, and provide evidence of how it unfolds across model layers. Beyond improving our understanding of the internal

mechanisms driving LLMs, our work lays the foundation for practical applications, particularly in optimizing token management and reducing computational costs.

2 RELATED WORK

Tokenization Sub-word tokenization algorithms (Wu, 2016; Kudo, 2018) are the standard for pre-processing text in modern LLMs. The most widely used method is byte-pair encoding (BPE; Senrich et al., 2016), which keeps frequent words intact and splits rare ones into multiple sub-words. Recent studies proposed ways to improve tokenization to consider word structure (Provilkov et al., 2020; Hofmann et al., 2022; Yehezkel & Pinter, 2023; Bauwens & Delobelle, 2024) or analyze how tokenization affects model performance (Bostrom & Durrett, 2020; Church, 2020; Klein & Tsarfaty, 2020; Zouhar et al., 2023; Schmidt et al., 2024). To the best of our knowledge, we are the first to thoroughly investigate how LLMs internally reconstruct word representations.

Detokenization and stages of inference Early LLM layers have been shown to integrate local context and map raw token embeddings into representations of concepts or entities—a process called *detokenization*. However, such observations were based on specific case studies (Elhage et al., 2022; Lad et al., 2024). More generally, recent work showed early layers provide local syntactic information (Tenney et al., 2019; Vulić et al., 2020; Durrani et al., 2020; Sajjad et al., 2022) or focus on extracting information from previous tokens (Ben Artzy & Schwartz, 2024). Ferrando & Voita (2024) analyzed token attributions in LLMs, observing attention heads that promote sub-word merging (Correia et al., 2019). Our work focuses on word-level detokenization, and goes beyond previous efforts to provide an in-depth analysis of how word representations are assembled from multiple tokens.

Interpreting the residual stream Recent methods for interpreting the intermediate states of LLMs draw on a *residual stream* perspective: the hidden state acts as an information stream along the layers, from which information is read at each layer, and new information is added through residual connections (Elhage et al., 2022). Thus, hidden states at any layer can be projected into the model’s vocabulary space, treating the hidden state as if it were the output of the last layer (nostalgebraist, 2020; Dar et al., 2023; Belrose et al., 2023; Yom Din et al., 2024). Similarly, Ghandeharioun et al. (2024) proposed to decode information from hidden representations into natural language, by patching (Zhang & Nanda, 2024) it into a prompt that encourages the model to *verbalize* the encoded information. We use both approaches to inspect how token representations evolve across layers.

LLM memories The idea of an *inner lexicon* aligns with recent work showing feedforward networks (FFN) layers in transformers act as key-value memories that encode factual and linguistic knowledge (Geva et al., 2021; Meng et al., 2022b; Dai et al., 2022). Particularly, FFNs were shown to enrich entity tokens with associated information (Meng et al., 2022b; Geva et al., 2023) and promote relevant concepts in vocabulary space to build up predictions (Geva et al., 2022). Our work expands on these findings, showing that word representations are pulled from FFN layers before emerging in the hidden state of the word’s last token.

Inner lexicon structure Inspired by studies on how concepts are encoded in LLMs (Park et al., 2024; 2025) and gradually promoted throughout their layers (Geva et al., 2022; Merullo et al., 2024), we consider the *inner lexicon* a “soft” lexicon, which (a) combines multiple vectors to form word representations (rather than a key-value dictionary); and (b) is not unique, i.e., a word might be stored and retrieved in more than one layer. Concurrently, Feucht et al. (2024) present evidence for an implicit vocabulary in LLMs, showing that models “forget” preceding tokens in multi-token words or multi-word entities, but remember previous tokens when processing single-token words.

3 A MOTIVATING OBSERVATION: LLMs CAN TELL WORDS FROM NON-WORDS

One of our key hypotheses in this work is that LLMs hold an internal lexicon of words, which is different from the BPE lexicon. We begin by asking whether LLMs, when processing a sequence of tokens, capture some notion of whether or not this sequence forms a word.